

Examen réparti 1 4I401(NOYAU)- Master Informatique

Novembre 2015

2 heures – Tout document papier autorisé

Barème donné à titre indicatif

I. COMMUTATION ET SAUTS (8 POINTS)

Les instructions *setjmp* et *longjmp* permettent de créer des continuations de code et sont utilisées pour réaliser des commutations entre threads d'un même processus. La fonction *swtch()* du noyau permet, elle, de réaliser la commutation entre processus différents.

I.1. (1 point)

Expliquez les différences entre ces deux mécanismes de commutation.

Setjmp/longjmp : rupture de séquence et sauvegarde uniquement de PC et SP ; il faut une sauvegarde explicite de pile et le *longjmp* modifie explicitement PC.

Swtch() : commutation faite sur retour de fonction. Entrée ds *swtch* sur pile de A et sortie sur pile de B, sans rupture de séquence dans *swtch*, c'est le retour sur *swtch* qui implique la modification de PC. Il y a en interne de *swtch* les appels à *savu* / *retu* qui sauvegarde les registres dans la zone u. Les zones u des différents processus étant distinguées (chacune étant dans son image swappable propre), il n'y a pas de mélange de pile.

On considère le code suivant :

```
#include<setjmp.h>
#include<stdio.h>
```

```
jmp_buf buf;
int f(int i) {
    while (1){
        printf("f(%d)\n",i);
        longjmp(buf,2);
    }
}
```

```
int g(int i) {
    while (1){
        printf("g(%d)\n",i);
        if(i>1)
            longjmp(buf,1);
    }
}
```

```
int main(int argc, char** argv) {

    switch(setjmp(buf)) {
        case 0:
            longjmp(buf,1);
            break;
        case 1:
            f(argc);
            break;
        default:
            g(argc);
            break;
    }

    printf("bye\n");
    return 0;
}
```

I.2. (2 points)

L'exécutable s'appelle **exo_commut**. Donnez l'affichage produit par **./exo_commut** et par **./exo_commut f g**. Dans les 2 cas, justifiez.

```

ema@biniou 15-16]$ ./exo_commut
f(1)
g(1)
g(1) ...

[ema@biniou 15-16]$ ./exo_commut f g
f(3)
g(3)
f(3)
g(3)
f(3)
g(3)
...

```

I.3. (1 point)

Peut-il y avoir confusion de piles entre **f** et **g** ? Justifiez.

Le point de sauvegarde est dans le main et chaque longjmp ramène dans la pile du main, ensuite il y a un appel soit à f soit à g, donc pas de mélange de pile.

I.4. (1 point)

Que se passe-t-il si on laisse s'exécuter le processus lancé par l'appel `./exo_commut f g` ?
Le programme boucle indéfiniment sans débordement de pile.

I.5. (1 point)

Expliquez pourquoi, dans cet exemple précis, si l'on connaît la taille de la pile des fonctions f et g (*stack_size*) il est possible de se passer d'une variable *top_stack* indiquant le haut de la pile ?

Car les 2 piles sont alignées : elles « partent » du même point (main) et font la même taille.

I.6. (2 points)

Modifiez le code de **f** et **g** *sans toucher au code du main* pour réaliser une commutation entre les deux fonctions à la fin de chaque itération. Vous pourrez introduire différentes variables supplémentaires ; vous justifierez leur emploi.

Il faut 2 buffers distincts (en + de buf qui ne peut être réemployé si on ne modifie pas le main, ce qui est une contrainte de l'énoncé) + sauvegarde de la pile de chaque fonction (sauvegarde brutale de taille fixe car pas de possibilité d'utiliser une var locale pour le sommet de pile ds main) . On retombe sur un code très proche de celui du TD, il faut penser à faire un setjmp (+ sauvegarde) en entrée de chaque fonction (sur son buffer dédié) :

(stack_size = 64 ici)

```

#include<setjmp.h>
#include<stdio.h>

```

```

jmp_buf buf, buff_f, buff_g;
unsigned int pile_f[64], pile_g[64];

int f(int i) {
    int n = 0;
    if(!setjmp(buff_f)){
        memcpy((unsigned int*) pile_f, (unsigned int *) &n, 64);
        longjmp(buf,2);
    }
    else
        memcpy((unsigned int *)&n, (unsigned int *)pile_f, 64);

    while(1){
        printf(" f(),  %d\n",n++ );
        if(!setjmp(buff_f)){
            memcpy((unsigned int*) pile_f, (unsigned int *) &n, 64);
            longjmp(buff_g,1);
        }
        else
            memcpy((unsigned int *)&n, (unsigned int *)pile_f, 64);
    }
    return 0;
}

int g(int i) {
    int m = 100;
    setjmp(buff_g); // sauvegarde pas indispensable ici
    while(1){
        printf("g(),  %d\n",m++);
        if(!setjmp(buff_g)){
            memcpy((unsigned int*) pile_g, (unsigned int *)&m,64);
            longjmp(buff_f,1);
        }
        else
            memcpy((unsigned int *)&m, (unsigned int *)pile_g,64);
    }
    return 0;
}

int main(int argc, char** argv) {

    switch(setjmp(buf)) {
    case 0:
        longjmp(buf,1);
        break;
    case 1:
        f(argc);
        break;
    default:
        g(argc);
        break;
    }

    printf("bye\n");
    return 0;
}

```

```

Son execution :
[ema@binou 15-16]$ ./exo_commut_v3
f(), 0
g(), 100
...
f(), 24350
g(), 24451
f(), 24351
g(), 24452
f(), 24352
g(), 24453
f(), 24353

```

II. SYNCHRONISATION (5 POINTS)

On souhaite implémenter un nouvel appel système `_waitincore` qui attend qu'un processus swappé (i.e., avec le flag `SLOAD` non positionné dans `p_flag`) soit ramené en mémoire. L'appel système `_waitincore` prend en paramètre d'entrée le `pid` du processus attendu. Si le processus n'est pas en mémoire, l'appelant doit s'endormir avec la priorité `PWAITINCORE`

II.1. (3 points)

Donnez le code de l'appel système `_waitincore`.

```

_waitincore() {
    int pid;
    struct proc *p;
    pid = u . u_arg [0] ;
    for ( p = &proc[0] ; p < &proc[NPROC] ; p++) {
        if (p->p_pid == pid && !p->p_flag&SLOAD) {
            p->p_flag |= INCOREWANTED;
            while (!p->p_flag&SLOAD)
                sleep(p+3, PWAITINCORE);
            return;
        }
    }
    return;
}

```

II.2. (2 points)

La fonction interne `returnincore(struct proc *p)` est appelée pour réveiller les processus qui attendent que `p` soit ramené en mémoire.

- Quel processus appelle la fonction `returnincore` ?
- Donnez le code de cette fonction.

- le swapper (la fonction `sched`)
- :

```

returnincore(struct proc *p){
    if (p->p_flag&INCOREWANTED) {
        p->p_flag &= ~INCOREWANTED;
    }
}

```

```

        wakeup(p+3);
    }
}

```

III.TIMEOUT (7 POINTS)

On considère les *timeout* UNIX tels que vus en TD. On considère également l'extension vue en TME : un *c_cid*, identifiant unique, est attribué à chaque *timeout*.

On souhaite offrir deux nouvelles fonctionnalités :

- `int postpone_timeout(int id, int nb_ticks)`. Cette fonction noyau retarde l'exécution du *timeout* identifié par *id* de *nb_ticks* (strictement positif). La fonction retourne 1 si le *timeout* identifié par *id* a été trouvé et mis à jour (et potentiellement déplacé), 0 sinon.
- `int multi_timeout(void (*fun)(), void *arg, int nb_ticks)`. Cette fonction crée un *timeout* qui s'exécutera **tous les** *nb_ticks* ticks. La fonction retourne le *c_cid* du *timeout* créé. On suppose qu'il existe une variable globale *cid* qui représente la valeur du dernier *cid* donné, on considère que retourner *cid++* est correct. On ne se préoccupe pas de l'arrêt d'un *multi_timeout* (on suppose que *untimeout(cid)* sera appelée à terme).

Pour le code, on pourra reprendre la structure du TME : des extraits des fichiers *callo.h* et *var.h* sont donnés en annexe.

III.1. (4 points)

Donnez le code de la fonction `int postpone_timeout (int id, int nb_ticks)`.
Basé sur le code du TME (*untimeout*)

```

int postpone_timeout(int id, int nb_ticks) {
    struct callo* p1, *p2, *p3;
    int t;
    void (*fun)();
    void* arg;

    p1 = &(v.ve_callout[0]);

    /* recherche de l'identifiant */
    while((p1->c_func != 0) && (p1->c_cid != id))
        p1++;

    /* pas trouvé */
    if(p1->c_func == 0)
        return 0;
}

```

```

/* on a trouvé, on cherche la nouvelle place (elle est plus bas car nb_tick>0)*/
/* mise à jour de l'identifiant suivant (~untimeout)*/
if((p1+1)->c_func != 0) {
    (p1+1)->c_time += p1->c_time; // on ajoute au suivant ce que l'on enlève
}
fun = p1->c_func;
arg = p1->c_arg;

/* basé sur timeout */
t=p1->c_time+nb_tick; // (calcul du nouveau temps relatif)
p2=p1;
MASK_ALRM;
/* recherche d'un emplacement dans le vecteur */
while((p2->c_func != 0) && (p2->c_time <= t)) {
    t -= p2->c_time;
    p2++;
}
p2->c_time -= t; // le nouveau suivant

// Il reste à remonter le tableau de p2-1 (emplacement de l'insertion) à P1 (vide)
// potentiellement p2-1==P1, dans ce cas la boucle ne fait rien
p3=p2-1;
while(p3>p1) {
    p1->c_time = (p1+1)->c_time;
    p1->c_func = (p1+1)->c_func;
    p1->c_arg = (p1+1)->c_arg;
    p1->c_cid = (p1+1)->c_cid;
    p1++;
}
// on insère en p2-1
p2-1->c_time = t;
p2-1->c_func = fun;
p2-1->c_arg = arg;
p2-1->c_cid = id;
UNMASK_ALRM;

return 1;
}

```

III.2. (3 points)

On souhaite implémenter la fonction multi_timeout. Pour cela on ajoute à la structure callo le champ « int c_multi ».

(a) Donnez le code de la fonction int multi_timeout (void (*fun)(), void *arg, int nb_ticks).

(b) Donnez les éventuelles modifications à apporter à la fonction restart().

On ajoute un champ “c_multi” (on peut aussi mettre, un flag et une valeur : besoin de retenir le nombre de ticks de la période).

```

int multi_timeout(void (*fun)(), void* arg, int nb_ticks) {
    struct callo *p1, *p2;
    int t;

```

```

t = nb_ticks;
p1 = &(v.ve_callout[0]);

MASK_ALRM;
/* recherche d'un emplacement dans le vecteur */
while((p1->c_func != 0) && (p1->c_time <= t)) {
    t -= p1->c_time;
    p1++;
}

p1->c_time -= t;
p2 = p1;

/* déplacement à la fin du vecteur */
while(p2->c_func != 0)
    p2++;

/* mise à jour des callout postérieurs au nouveau */
while(p2 >= p1) {
    (p2+1)->c_time = p2->c_time;
    (p2+1)->c_func = p2->c_func;
    (p2+1)->c_arg = p2->c_arg;
    (p2+1)->c_cid = p2->c_cid;
    (p2+1)->c_multi = p2->c_multi;
    p2--;
}
p1->c_time = t;
p1->c_func = fun;
p1->c_arg = arg;
p1->c_cid = cid;
p1->c_multi = nb_ticks; // c_multi == 0 pour les non multi.

UNMASK_ALRM;

/* retourne l'identifiant du nouveau callout */
return cid++;
}

```

Dans restart, juste après l'appel à la fonction :

```

while(p1->c_func!=0 && p1->c_time<=0) {
    (*p1->c_func)(p1->c_arg);
    if (p1->c_func) { // c'est un multi
        multi_timeout(p1->c_func, p1->c_arg, p1->c_multi); // là il faut bien appeler multi, et
        // passer c_multi pas c_time (==0)
    }
    p1++;
}

```

Extrait du fichier callo.h

```
/**
 * @struct callo
 * Définition de la structure callo
 */
struct callo {
    int      c_time;      /**< incremental time */
    bc_caddr_t c_arg;      /**< argument to routine */
    void      (*c_func)(); /**< routine */
    int      c_cid;       /**< callout ID */
};

/**
 * @var extern int cid
 * Identifiant d'un callout
 */
extern int cid;

/**
 * @fn extern void restart(void)
 * Exécute les fonctions figurant dans le vecteur de callout dont le compteur est
inférieur ou égal à zéro
 */
extern void restart(void);

/**
 * @fn int timeout(void (*fun)(), void *arg, int tim)
 * Crée une nouvelle entrée dans le vecteur de callout et l'initialise avec les
paramètres suivants :
 * - fun
 * - arg
 * - tim
 * @param fun : fonction à appeler
 * @param arg : argument de la fonction fun
 * @param tim : durée en nombre de ticks après laquelle fun est appelée
 * @return L'identifiant du callout créé
 */
extern int timeout(void(*)(), void*, int);

/**
 * @fn void untimeout(int ident)
 * Permet la suppression d'un callout
 * @param ident : identifiant du callout à supprimer
 */
extern void untimeout(int);
```

Extrait du fichier var.h

```
/**
 * Nombre d'entrées dans le vecteur de callout
 */
#define MAX_CALLOUT 100
/**
 * struct v
 */
struct {
    int    v_hsize;
    int    v_hmask;
    struct callo ve_callout[MAX_CALLOUT];
}v;
```